

DOCUMENTAÇÃO DO PROJETO

ENGENHARIA DE SOFTWARE

Projeto Integrador V – Aplicativos Gamificados

The Academic Quest

Nome da Empresa (Grupo): The academic Quest

Curso: TAD / ENS

Período: Noturno

Instituição Parceira: UNIVAG

Professor Orientador: Brendo Vale

Semestre Letivo: 5º e 6º

EQUIPE:

#	Nome Completo	Função no Projeto	Modalidade
1	Arthur Di Loretto	Dev Backend	Ao vivo
2	João Gabriel	Dev Frontend	Ao vivo
3	Erik Beneviedes	Documentação	Ao vivo
4	Guilherme Sanches	UX/UI Designer	Ao vivo
5	Gabriel Beretta	QA / Tester	Ao vivo
6	William		

HISTÓRICO DE VERSÕES

Data	Versão	Descrição da Alteração	Autor
05/03/2026	1.0	Criação do documento e definição do problema	Arthur, Erik, João
17/03/2026	2.0	Elaboração da metodologia e requisitos do sistema	Erik
26/03/2026	3.0	Definição da Stack e Configuração CI/CD	Arthur, Erik, João

FASE 1: DESCOBERTA E DEFINIÇÃO (INCEPTION)

1.1 Contextualização e Problema

Muitos estudantes do ensino superior enfrentam dificuldades na organização e execução de Projetos Integradores e Trabalhos de Conclusão de Curso (TCC). Os principais problemas incluem falta de engajamento, dificuldades em compreender cronogramas acadêmicos, falta de divisão clara de responsabilidades nos grupos e baixa motivação para cumprir prazos.

Isso frequentemente resulta em atrasos, trabalhos de baixa qualidade e até evasão acadêmica. Professores também enfrentam dificuldade para acompanhar o progresso individual dos alunos dentro de trabalhos em grupo.

O aplicativo The Academic Quest propõe transformar o processo acadêmico em uma experiência gamificada inspirada em RPG, onde tarefas viram missões, equipes viram clãs e progresso acadêmico se torna evolução dentro de um jogo.

1.2 Público-Alvo e Personas

Persona	Perfil	Dor Principal	Necessidade
Lucas, 21 anos	Estudante de TAD	Dificuldade em organizar entregas da universidade	Visualizar claramente prazos e responsabilidades
Mariana, 24 anos	Estudante de ADM	Dificuldade em entender o valor das entregas	Se manter motivado durante os estudos
Carlos, 42 anos	Professor de Contabilidade	Dificuldade em acompanhar a evolução dos grupos e contextualização dos projetos	Visualizar a evolução e trabalhos dos grupos de forma facilitada

1.3 Alinhamento com os ODS (ONU)

ODS Principal: ODS 4 – Educação de Qualidade

Justificativa: O aplicativo contribui para melhorar a qualidade da educação ao aumentar o engajamento dos estudantes em atividades acadêmicas.

Ao transformar tarefas educacionais em desafios gamificados, o sistema incentiva colaboração, disciplina e participação ativa no aprendizado.

1.4 Validação e Vídeo de Pitch

1.4.1 Pesquisa de Mercado

Para validar a relevância do problema identificado, foi realizada uma pesquisa com estudantes do ensino superior que participam de disciplinas como Projeto Integrador e Trabalho de Conclusão de Curso.

A pesquisa teve como objetivo entender:

- dificuldades na organização de projetos acadêmicos em grupo
- nível de motivação dos alunos
- ferramentas utilizadas atualmente para organização
- interesse em soluções gamificadas

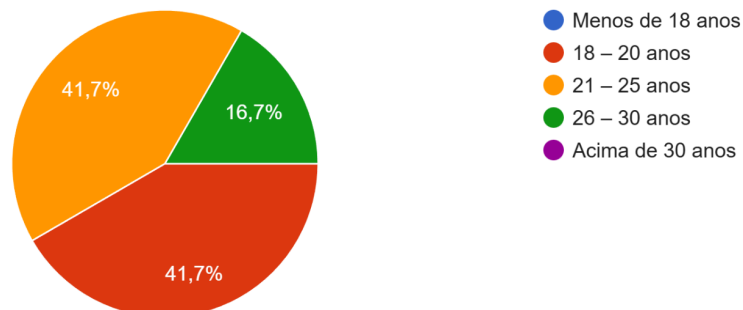
Metodologia da Pesquisa

- Tipo de pesquisa: Questionário online
- Ferramenta: Google Forms
- Público entrevistado: Estudantes universitários
- Quantidade de respostas: 12 estudantes
- Cursos: Engenharia de software, Sistemas de informação, Análise e Desenvolvimento de sistemas, Ciências da Computação, etc.

Resultados Principais

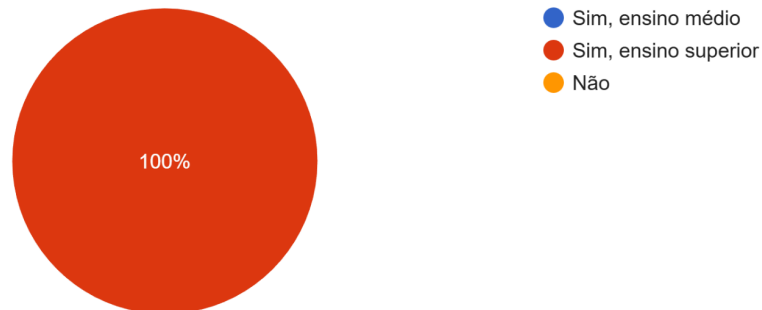
1. Qual sua idade?

12 respostas



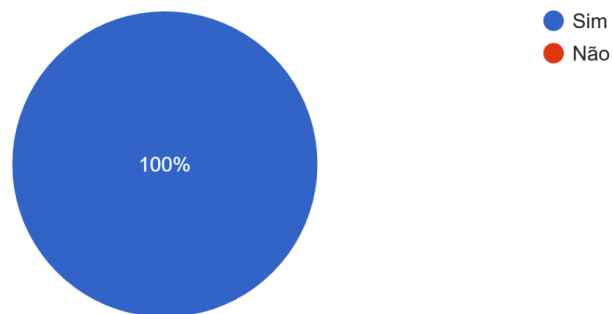
2. Você é estudante?

12 respostas



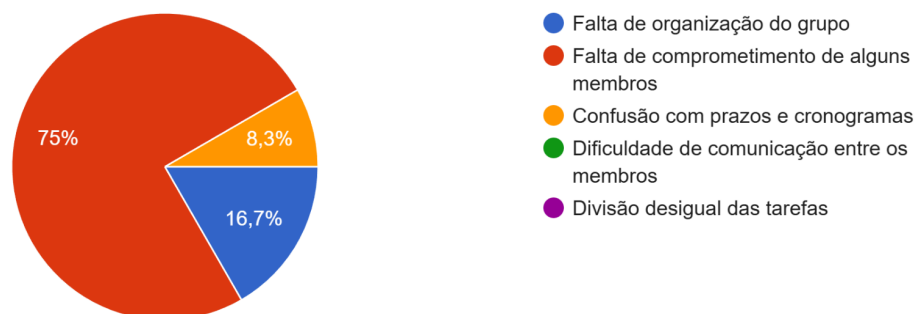
4. Você já participou de algum trabalho acadêmico em grupo (Projeto Integrador, TCC, trabalhos de disciplina etc.)?

12 respostas



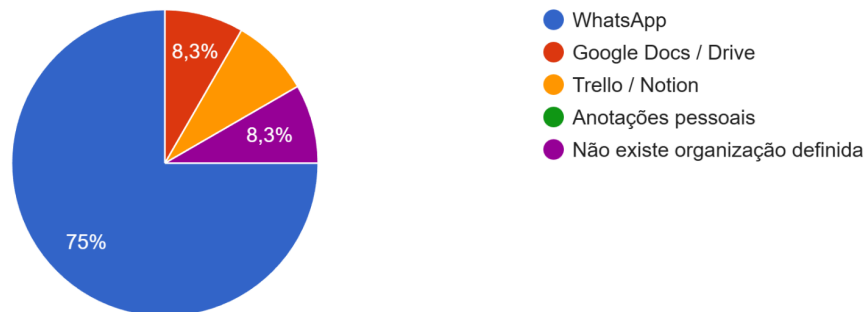
5. Qual a maior dificuldade que você já enfrentou em trabalhos acadêmicos em grupo?

12 respostas



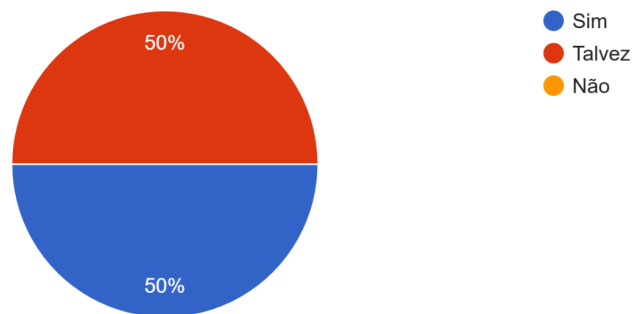
6. Normalmente, como você organiza tarefas de trabalhos em grupo?

12 respostas



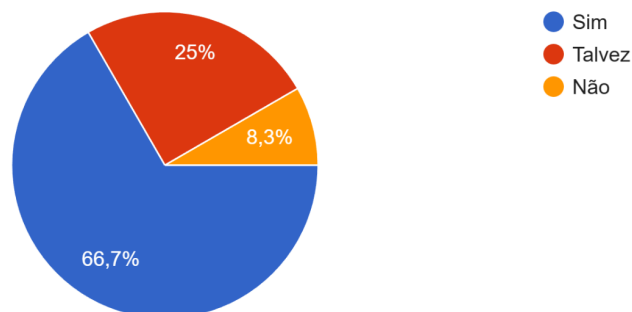
7. Você acredita que elementos de jogos podem ajudar a tornar atividades acadêmicas mais motivadoras?

12 respostas



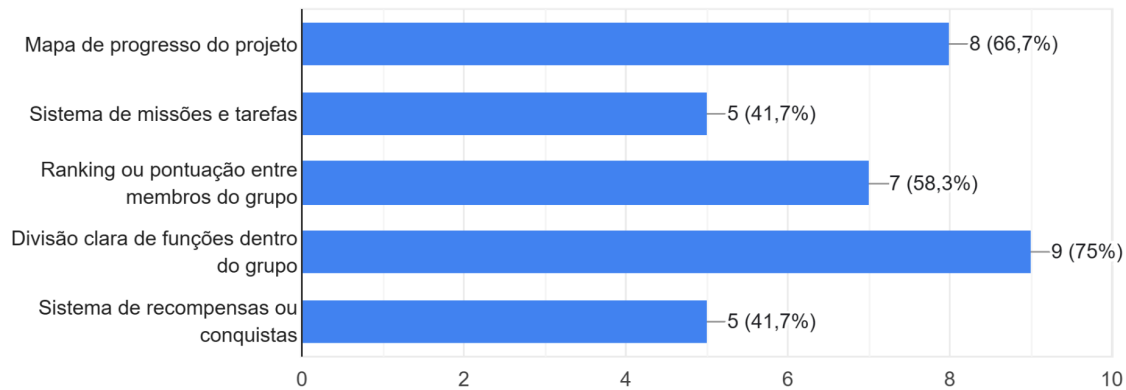
8. Você utilizaria um aplicativo que transforma tarefas acadêmicas em missões de um jogo (com níveis, XP, rankings e progresso)?

12 respostas



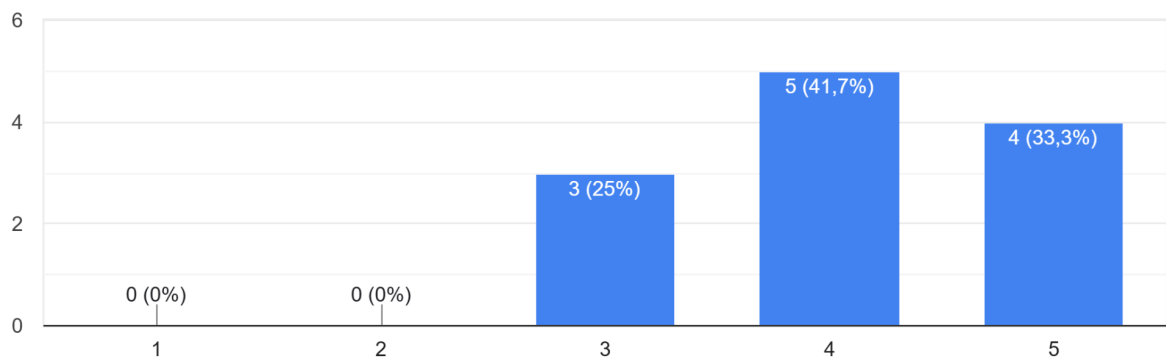
9. Qual dessas funcionalidades você acharia mais interessante em um aplicativo para organizar projetos acadêmicos?

12 respostas



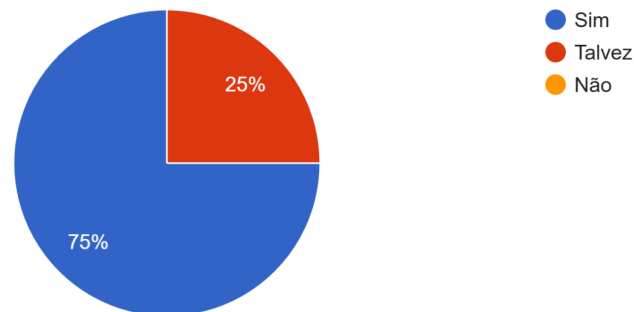
10. Você acha que um aplicativo assim ajudaria estudantes a se organizarem melhor em projetos acadêmicos?

12 respostas



11. Você usaria um aplicativo assim se estivesse disponível gratuitamente?

12 respostas



Conclusão da Validação

Os resultados indicam que a maioria dos estudantes enfrenta dificuldades relacionadas à organização e colaboração em projetos acadêmicos. Além disso, houve alto interesse em uma solução que utilize gamificação para aumentar engajamento e organização, validando a proposta do aplicativo The Academic Quest.

Link do Vídeo (YouTube): [The Academic Quest - Apresentação \(pitch\)](#)

1.5 Checklist de Entrega da Fase 1

- [X] - Problema claramente definido e contextualizado
- [X] - Pelo menos 2 personas criadas
- [X] - ODS identificado com justificativa
- [X] - Vídeo de pitch gravado e publicado

FASE 2: ENGENHARIA E DESIGN (ELABORAÇÃO)

2.1 Metodologia de Desenvolvimento

Metodologia escolhida : Scrum

Duração dos sprints: 2 semanas

Ferramentas de gestão: Trello – gerenciamento de tarefas; GitHub – versionamento de código; Discord / WhatsApp – comunicação da equipe

A equipe realizará reuniões semanais de planejamento (Sprint Planning), acompanhamento rápido das tarefas (check-ins) e retrospectivas ao final de cada sprint para melhoria contínua.

2.2 Requisitos do Sistema

Definição de Atores:

Usuário: entidade genérica do sistema que representa qualquer pessoa que utiliza a aplicação.

Aluno: tipo de usuário que participa de grupos, realiza missões e acompanha seu progresso.

Líder de Grupo: um Aluno com permissões adicionais, responsável por criar grupos, gerenciar membros, criar missões e validar missões concluídas.

Professor: usuário responsável por acompanhar o desempenho dos grupos e projetos, sem participação direta na execução das missões.

2.2.1 Requisitos Funcionais (RF)

ID	Descrição (SMART)	Prioridade	Ator Principal	Sprint
RF01	O sistema deve permitir o cadastro de usuários com nome, e-mail e senha	Essencial	Usuário	1
RF02	O sistema deve permitir login de usuários com validação de credenciais	Essencial	Usuário	1
RF03	O sistema deve permitir a criação de grupos (clãs) para organização de projetos acadêmicos	Essencial	Líder de Grupo	1
RF04	O sistema deve permitir entrada de usuários em grupos	Essencial	Aluno	1

ID	Descrição (SMART)	Prioridade	Ator Principal	Sprint
RF05	O sistema deve permitir o gerenciamento de grupos (adicionar/remover membros)	Essencial	Líder de Grupo	1
RF06	O sistema deve permitir o gerenciamento de grupos (adicionar/remover membros)	Essencial	Líder de Grupo	1
RF07	O sistema deve permitir a conclusão de missões pelos usuários	Essencial	Aluno	2
RF08	O sistema deve permitir a conclusão de missões pelos usuários	Essencial	Líder de Grupo	2
RF09	O sistema deve permitir a validação de missões concluídas	Importante	Líder de Grupo	2
RF10	O sistema deve permitir a visualização do progresso do projeto	Importante	Usuário	2
RF11	O sistema deve permitir a atribuição de classes aos membros do grupo	Importante	Líder de Grupo	2
RF12	O sistema deve permitir que professores acompanhem o progresso dos grupos	Importante	Professor	2

2.2.2 Requisitos Não Funcionais (RNF)

ID	Descrição (SMART)	Categoria	Métrica de Aceite
RNF01	O sistema deve responder às ações do usuário em menos de 2 segundos	Desempenho	Tempo de resposta < 2s
RNF02	O sistema deve utilizar autenticação segura baseada em serviços confiáveis (ex: Firebase Authentication), garantindo a proteção dos dados dos usuários	Segurança	Autenticação implementada com criptografia e controle de acesso
RNF03	O aplicativo deve possuir interface simples e intuitiva para facilitar o uso pelos estudantes	Usabilidade	Usuários conseguem realizar tarefas básicas sem treinamento

ID	Descrição (SMART)	Categoria	Métrica de Aceite
RNF04	O sistema deve ser compatível com dispositivos Android com versão 8.0 ou superior	Compatibilidade	Funcionamento testado em dispositivos Android

2.2.3 Requisitos de Domínio (RD)

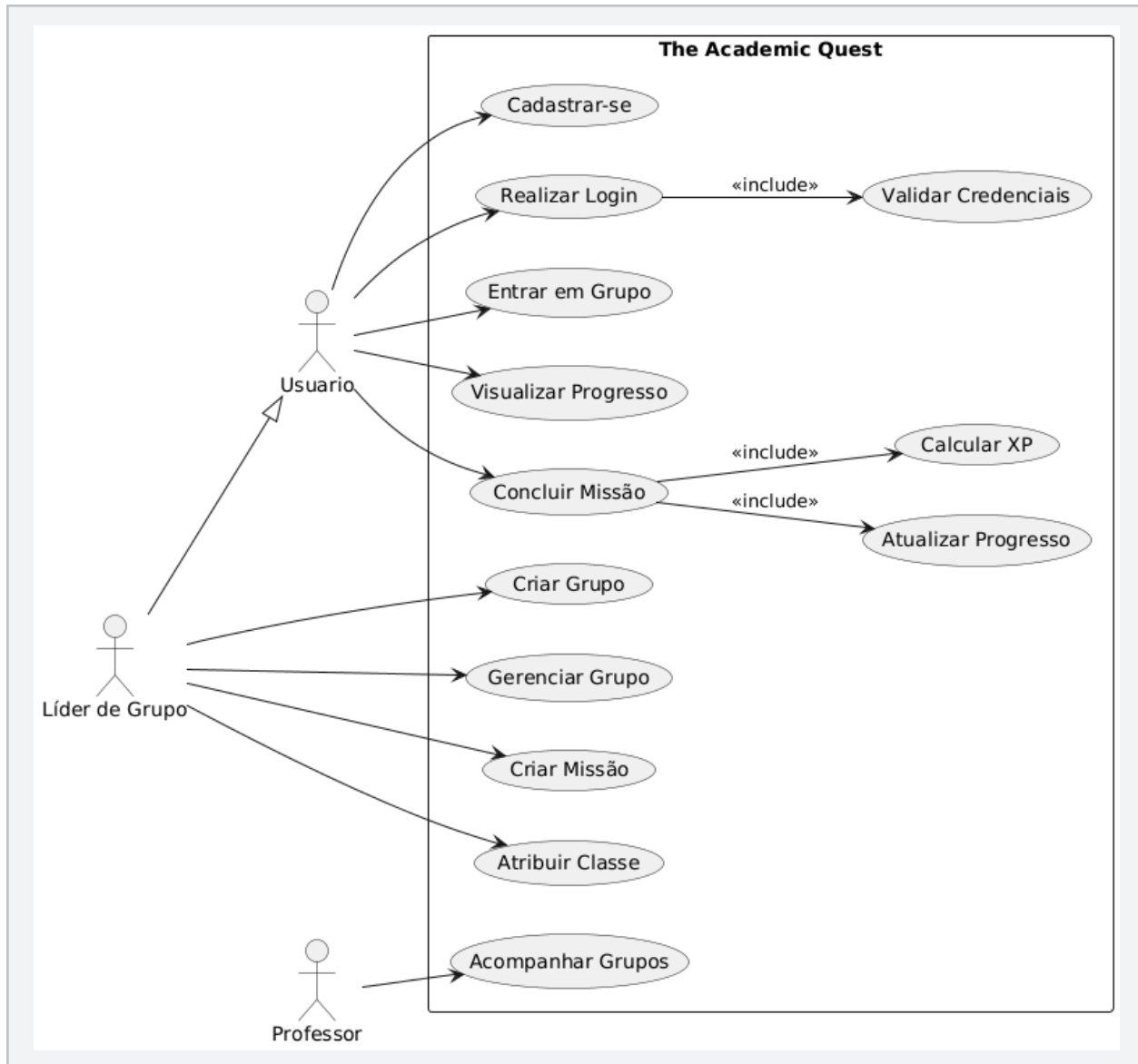
ID	Descrição	Origem/Regra	Impacto no Sistema
RD01	Cada projeto acadêmico deve possuir um prazo final definido pelo professor ou líder do grupo	Regras acadêmicas	O sistema deve permitir configurar datas limite para missões e entregas
RD02	Cada membro do grupo deve possuir uma função (classe) específica dentro do projeto	Mecânica de gamificação	O sistema deve permitir atribuir classes como Escriba, Paladino ou Ranger
RD03	O progresso do projeto deve ser medido pela conclusão das tarefas (missões)	Mecânica de progressão	O sistema deve calcular automaticamente o progresso geral do projeto
RD04	A experiência (XP) dos usuários deve ser concedida somente após a conclusão e validação de tarefas	Sistema de recompensas	O sistema deve registrar XP e evolução do usuário após validação da missão
RD05	Um usuário só pode participar de um grupo de projeto por disciplina ou projeto ativo	Organização acadêmica	O sistema deve restringir múltiplas participações no mesmo projeto

2.2.4 Histórias de Usuário (User Stories)

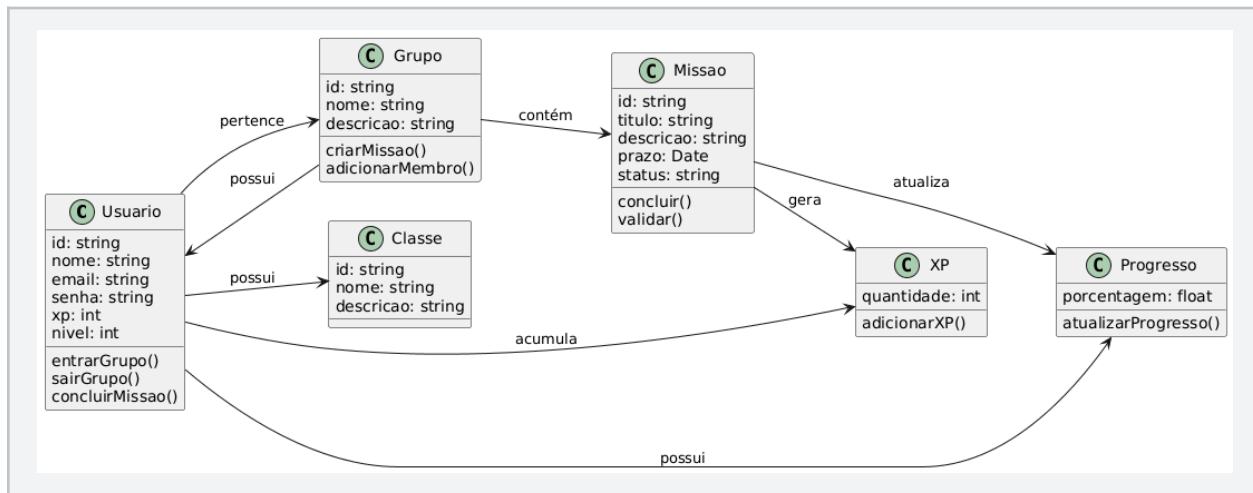
#	Como... (Quem)	Eu quero... (O Quê)	Para... (Por Quê)
US01	Aluno	Me cadastrar no sistema	Acessar a plataforma
US02	Aluno	Fazer login com meu e-mail e senha	Acessar minhas informações
US03	Aluno	Entrar em um grupo de projeto	Participar das atividades em equipe
US04	Líder de Grupo	Criar um grupo	Organizar minha equipe de projeto
US05	Líder de Grupo	Adicionar membros ao grupo	Formar uma equipe colaborativa
US06	Líder do grupo	Criar missões com título, descrição e prazo	Organizar as tarefas do projeto
US07	Aluno	Concluir uma missão	Registrar minha contribuição no projeto
US08	Líder de grupo	Validar uma missão concluída	Registrar minha contribuição no projeto
US09	Aluno	Receber XP ao concluir missões validadas	Acompanhar minha evolução no sistema
US10	Aluno	Visualizar o progresso do projeto	Acompanhar minha evolução no sistema
US11	Líder de Grupo	Atribuir classes aos membros	Definir responsabilidades na equipe
US12	Professor	Visualizar o progresso dos grupos	Acompanhar o desempenho dos alunos

2.3 Modelagem de Software (UML)

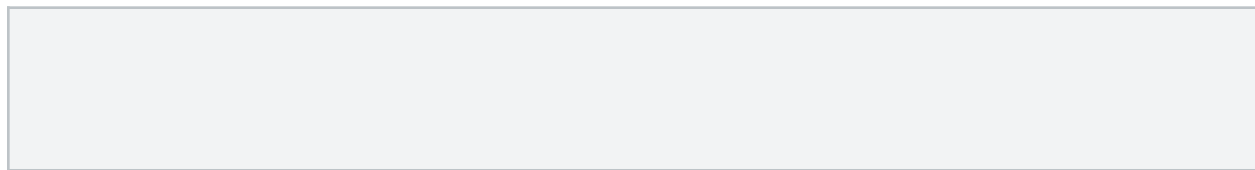
2.3.1 Diagrama de Casos de Uso



2.3.2 Diagrama de Classes / Entidade-Relacionamento



2.3.3 Diagrama de Sequência (Opcional, mas recomendado)



2.4 Arquitetura e Padrões

Padrão Arquitetural: MVC (Model–View–Controller)

O padrão MVC permite separar três responsabilidades principais do sistema:

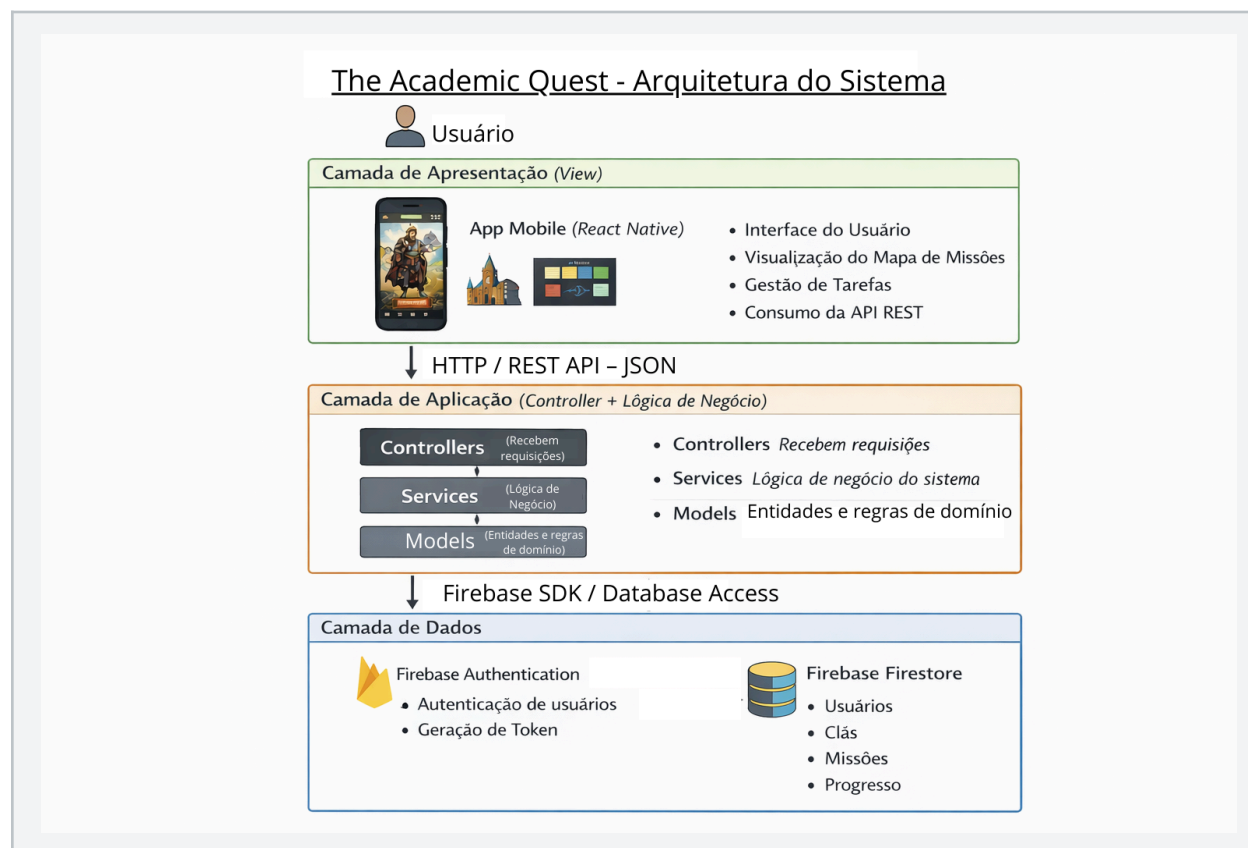
Model: responsável pelo gerenciamento dos dados da aplicação, como usuários, grupos, missões e progresso.

View: responsável pela interface do usuário no aplicativo mobile.

Controller: responsável pela lógica de negócio e comunicação entre a interface e os dados.

Essa separação facilita manutenção do código, organização do desenvolvimento, escalabilidade da aplicação e trabalho colaborativo entre membros da equipe

2.4.1 Diagrama de Arquitetura



2.5 Gestão de Riscos

#	Risco	Probabilidade	Impacto	Plano de Mitigação
R01	Atraso no desenvolvimento do aplicativo	Média	Alto	Dividir tarefas em sprints menores
R02	Falta de experiência da equipe com algumas tecnologias	Alta	Médio	Pesquisa e estudo da documentação das ferramentas
R03	Baixo engajamento inicial de usuários	Baixa	Alto	Investir em gamificação e interface atrativa
R04	Problemas de integração entre frontend e backend	Baixa	Médio	Testes frequentes durante o desenvolvimento

2.6 Planejamento Estratégico e Viabilidade

2.6.1 Modelo de Negócio

Modelo escolhido (SaaS, B2B, Freemium, Licenciamento, Open Source, etc.): Freemium

O aplicativo poderá ser utilizado gratuitamente por estudantes com funcionalidades básicas de organização de projetos.

Instituições de ensino poderão adquirir versões com funcionalidades adicionais, como: relatórios de desempenho; acompanhamento de turmas; dashboards para professores; integração com sistemas acadêmicos.

2.6.2 Canvas Simplificado do Projeto

Proposta de Valor	Segmento de Clientes	Canais
Auxiliar estudantes a organizar projetos acadêmicos de forma motivadora através de mecânicas de gamificação inspiradas em RPG	Estudantes universitários, professores orientadores, instituições de ensino	Aplicativo mobile, divulgação em universidades, redes sociais
Fontes de Receita	Estrutura de Custos	Métricas-Chave
Assinaturas institucionais, planos premium para instituições, possíveis parcerias educacionais	Infraestrutura em nuvem, manutenção do sistema, desenvolvimento contínuo	Número de Usuários Ativos; Número de Projetos Criados; Taxa de Conclusão de Missões; Engajamento dos Usuários; Taxa de Conclusão de Projetos; Satisfação dos Usuários

2.6.3 Investimento Inicial (CapEx)

Observação: O projeto não possui custo financeiro direto, pois utiliza ferramentas gratuitas e de código aberto, como React Native, Node.js e Firebase.

Entretanto, há investimento indireto relacionado ao tempo da equipe, aprendizado das tecnologias e desenvolvimento do sistema. Além disso, o uso de serviços em nuvem pode gerar custos futuros conforme o crescimento da aplicação.

O projeto é desenvolvido por uma equipe de 6 integrantes, com dedicação contínua ao longo das sprints.

Recurso	Descrição	Valor (R\$)
Desenvolvimento	Criação do aplicativo	R\$ 0,00*
Design	Interface e prototipagem	R\$ 0,00*
Infraestrutura	Configuração de serviços em nuvem	R\$ 0,00*
TOTAL		R\$ 0,00*

*Custos financeiros diretos não aplicáveis no contexto acadêmico.

2.6.4 Custo Operacional Mensal (OpEx)

Observação: Durante a fase acadêmica, o sistema utilizará planos gratuitos (free tier) dos serviços em nuvem. Em um cenário real de produção, os custos podem variar conforme o número de usuários e volume de dados.

Serviço	Finalidade	Valor Mensal (R\$)
Hospedagem	Infraestrutura em nuvem	R\$ 0,00*
Banco de Dados	Armazenamento de dados	R\$ 0,00*
Manutenção	Atualizações e correções	R\$ 0,00*
TOTAL		R\$ 0,00*

*Valores baseados no uso de planos gratuitos (free tier).

2.6.5 Cronograma de Entregas

Fase	Entrega Esperada	Atividades Principais	Prazo Final
Fase 1	Validação do Problema	Pesquisa, Personas, Pitch	[06/03/26]
Fase 2	Design e Planejamento	Requisitos, UML, Arquitetura	[13/03/26]
Fase 3	Versão Alpha	Desenvolvimento e CI/CD	[01/05/26]
Fase 4	Lançamento e Testes	Testes com usuários, ajustes finais	[01/06/26]

Checklist de Entrega da Fase 2

- [X] - Metodologia definida com ferramentas e ritos
- [X] - Pelo menos 5 Requisitos Funcionais documentados
- [X] - Requisitos Não Funcionais e de Domínio preenchidos
- [X] - User Stories escritas para os RFs principais
- [X] - Diagramas UML criados (Casos de Uso + Classes/ER)
- [X] - Arquitetura definida e justificada
- [X] - Pelo menos 4 riscos identificados com planos de mitigação
- [X] - Cronograma e viabilidade financeira preenchidos

FASE 3: CONSTRUÇÃO E QUALIDADE (IMPLEMENTAÇÃO)

3.1 Stack Tecnológica

Camada	Tecnologia	Justificativa
Frontend	React Native	Permite o desenvolvimento de aplicativo mobile multiplataforma, com reaproveitamento de código para Android e iOS.
Frontend	Expo	Facilita execução, testes e build do aplicativo mobile
Frontend	TypeScript	Adiciona tipagem estática ao projeto, reduzindo erros e facilitando a manutenção do código.
Backend	Node.js	Ambiente de execução leve e eficiente para desenvolver a lógica do servidor e da API.
Backend	Express.js	Framework simples para criação de APIs
Backend	REST API	Comunicação padronizada entre frontend e backend
Banco de Dados	MongoDB	Banco NoSQL escalável e fácil integração com mobile
Autenticação	Firebase Authentication	Sistema seguro e pronto para login de usuários
Versionamento	Git	Controle de versão distribuído
Repositório	GitHub	Armazenamento e colaboração no código
CI/CD	GitHub Actions	Automação de build e testes
Build Mobile	Expo EAS	Geração de builds do aplicativo
Comunicação	WhatsApp / Discord	Comunicação da equipe
Testes	Jest	Testes unitários em JavaScript
API Testes	Postman	Testes de endpoints da API

3.2 Pipeline de CI/CD

O projeto utilizará um pipeline de Integração Contínua e Entrega Contínua (CI/CD) para automatizar o processo de build e validação do sistema.

O fluxo será estruturado da seguinte forma:

O código será hospedado no GitHub, onde todos os membros da equipe irão versionar suas alterações utilizando o modelo Git Flow.

As branches serão organizadas da seguinte forma:

- main: versão estável do sistema
- develop: versão em desenvolvimento
- feature/*: novas funcionalidades

A cada push ou pull request para a branch develop:

- O GitHub Actions será acionado automaticamente
- Serão executados testes automatizados com Jest
- Será feita a verificação de erros no código

Para builds do aplicativo mobile:

- Será utilizado o Expo EAS para gerar versões do app
- Builds poderão ser gerados para testes em dispositivos reais

O deploy será realizado da seguinte forma:

- Backend: hospedado em ambiente de nuvem (ex: Firebase ou serviço compatível)
- Frontend: distribuído via build do Expo

Esse pipeline garante maior qualidade, integração contínua e redução de erros durante o desenvolvimento.

3.3 Repositório e Documentação

Link do GitHub: [Repositório The Academic Quest](#)

3.3.1 Manual de Instalação

Para executar o projeto localmente, siga os passos abaixo:

1. Pré-requisitos

- Node.js instalado
- NPM ou Yarn
- Expo CLI instalado globalmente
- Conta no Firebase (para configuração do backend)

2. Clonar o repositório

```
git clone https://github.com/seu-repositorio/aqui.git  
cd academic-quest
```

3. Instalar dependências

```
npm install
```

ou

```
yarn install
```

4. Configurar variáveis de ambiente

Criar um arquivo .env na raiz do projeto e adicionar:

```
FIREBASE_API_KEY=...  
FIREBASE_AUTH_DOMAIN=...  
FIREBASE_PROJECT_ID=...
```

5. Executar o projeto

```
npx expo start
```

O aplicativo poderá ser executado:

- Em emulador Android
- Em dispositivo físico via QR Code

3.4 Padrões de Código e Boas Práticas

Aspecto	Convenção Adotada
Nomeação de variáveis	camelCase
Nomeação de classes/componentes	PascalCase
Organização de arquivos	Estrutura por funcionalidade
Separação de responsabilidades	Uso do padrão MVC
Funções	Funções pequenas e com responsabilidade única
Comentários	Apenas quando necessário, código autoexplicativo
Controle de versão	Commits pequenos e descritivos
Branches	Padrão Git Flow
Tratamento de erros	Uso de try/catch
Padronização de código	Uso de ESLint

Checklist de Entrega da Fase 3

- [X] - Stack tecnológica definida e justificada
- [X] - Repositório GitHub com commits regulares da equipe
- [X] - Pipeline de CI/CD configurado (mesmo que básico)
- [X] - Manual de instalação funcional
- [] - Versão Alpha do app funcional

FASE 4: VALIDAÇÃO E ENTREGA (TRANSIÇÃO)

INFO O QUE É ESTA FASE?: O software está pronto? Só os usuários reais podem confirmar. Nesta fase, vocês testam com pessoas de verdade, medem o impacto e documentam os resultados.

4.1 Estratégia de Testes

Teste é garantia de qualidade. Defina quais tipos de testes serão aplicados e como.

4.1.1 Testes de Software

Tipo de Teste	O Que Testa	Ferramenta/Abordagem
Unitário	Funções e componentes isolados	Jest, React Testing Library
Integração	Comunicação entre módulos	Supertest, Firebase Emulators
Manual/E2E	Fluxos completos do usuário	Roteiro de testes manuais
[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]

4.1.2 Testes de Usabilidade

Sessões com usuários reais da escola parceira para validar se o app é fácil de usar e resolve o problema.

#	Perfil do Testador	Tarefa Testada	Resultado	Observações
1	Aluno, 15 anos	Fazer login e ver notas	Sucesso em 30s	Botão muito pequeno
[Preencher]	[Preencher]	[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]	[Preencher]	[Preencher]

4.2 Indicadores de Impacto

Métricas concretas que provam que o projeto resolveu (ou contribuiu para resolver) o problema definido na Fase 1.

Indicador	Meta	Resultado Real	Evidência
Adesão dos alunos	> 70% da turma	[Preencher após testes]	Screenshot / Analytics
Satisfação (NPS)	> 8/10	[Preencher após testes]	Formulário Google
[Preencher]	[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]	[Preencher]

4.3 Lições Aprendidas

Reflitam sobre o processo: o que deu certo, o que fariam diferente, e o que aprenderam como engenheiros de software.

Descrevam as principais lições: desafios técnicos superados, dinâmica da equipe, relação com a escola parceira, habilidades desenvolvidas...

4.4 Conclusão

Escrevam uma conclusão conectando o problema da Fase 1 com os resultados da Fase 4. Como a engenharia de software ajudou a resolver o problema? O que o projeto trouxe de valor para a comunidade?

Checklist de Entrega da Fase 4

- [] - Testes de software executados e documentados
- [] - Testes de usabilidade realizados com usuários reais
- [] - Indicadores de impacto preenchidos com dados reais
- [] - Lições aprendidas documentadas
- [] - Conclusão final escrita
- [] - App funcional e demonstrável

REFERÊNCIAS

ATENÇÃO: Todo projeto com indícios de plágio será REPROVADO. Citem todas as documentações, tutoriais, artigos e bibliografias utilizados. Use o formato ABNT.

MEIRA, Luciano. Ludicidade, jogos digitais e gamificação na aprendizagem. Porto Alegre: Grupo A, 2019.

Adicionem aqui as referências do projeto. Incluem: documentações oficiais (React Native, Firebase, etc.), artigos, livros, tutoriais e qualquer material que tenha sido utilizado.

NOTA DO PROFESSOR

INFO ORIENTAÇÃO FINAL: Este template foi projetado para guiar vocês na criação de um projeto real de engenharia de software. Não se trata apenas de codificar um app, mas de projetar um sistema sustentável. Sigam cada fase com atenção, preencham com honestidade e lembrem-se: no mercado, vocês lidarão com documentos como SAD (Arquitetura), PRD (Requisitos) e Planos de Teste complexos. Este documento condensa o essencial com o rigor técnico esperado de um engenheiro de software.

Professor Orientador: Brendo Vale

Semestre: 2026/1